



DSA103 Advanced Chemical Data Science

Lecture 3: Python review part 1

Prof. Meredith C. Schuman

meredithchristine.schuman@uzh.ch

```
1 def search(self, state, g_score):
2     cube = RubiksCube(state=state)
3     if cube.solved():
4         return True
5     elif len(self.moves) >= self.threshold:
6         return False
7     min_val = float('inf')
8     best_action = None
9     for a in [(r, n, d) for r in ['h', 'v', 's'] for d in [0, 1] for n in range(cube.n)]:
10        cube = RubiksCube(state=state)
11        if a[0] == 'h':
12            cube.horizontal_twist(a[1], a[2])
13        elif a[0] == 'v':
14            cube.vertical_twist(a[1], a[2])
15        elif a[0] == 's':
16            cube.side_twist(a[1], a[2])
17        if cube.solved():
18            self.moves.append(a)
19            return True
20        cube_str = cube.stringify()
21        h_score = self.heuristic[cube_str] if cube_str in self.heuristic else self.max_depth
22        f_score = g_score + h_score
23        if f_score < min_val:
24            min_val = f_score
25            best_action = [(cube_str, a)]
26        elif f_score == min_val:
27            if best_action is None:
28                best_action = [(cube_str, a)]
29            else:
30                best_action.append((cube_str, a))
31        if best_action is not None:
32            if self.min_threshold is None or min_val < self.min_threshold:
33                self.min_threshold = min_val
34            next_action = choice(best_action)
35            self.moves.append(next_action[1])
36            status = self.search(next_action[0], g_score + min_val)
37            if status: return status
38        return False
```

Questions on the homework?



Now a brief python refresher

We'll rest in a “valley” reviewing basic Python concepts before continuing up the mountain. 😊

- Variable assignment and types
- Loops and iterables
- Data structures (e.g. lists, tuples, dictionaries)
- List comprehension
- Scope (local vs. global)
- Functions and modules
- Keywords



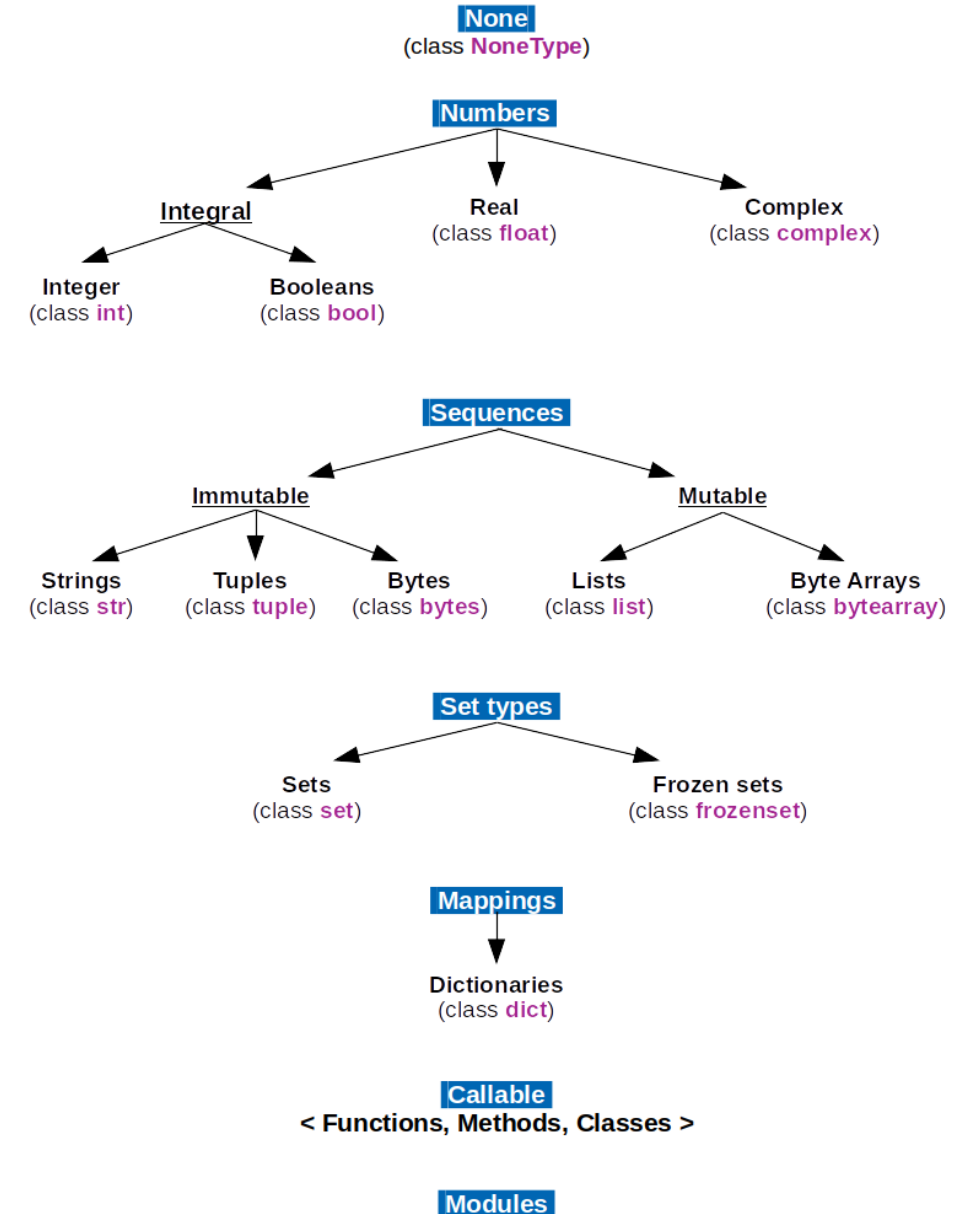
[Getty Images](#) for [Unsplash+](#)

Variable assignment, types, iterables, data structures

Python 3
The standard type hierarchy

Properties of Python types
(What is a type? See Lecture 2)

- Mutable or immutable
- Iterables
 - All sequence types
 - Some non-sequence types (dict, file objects)
 - Classes defined with an `__iter__()` method or with a `__getitem__()` method implementing sequence semantics
- What are appropriate data structures for a variable?



<https://docs.python.org/3/glossary.html#term-iterable>

https://commons.wikimedia.org/wiki/File:Python_3._The_standard_type_hierarchy.png by Максим Пе, CC-BY-SA 4.0

List comprehension

```
print([x for x in range(2,100) if all(x % y != 0 for y in range(3,x))])
```

✓ 0.0s

[2, 3, 4, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

Which is shorter?

Which is easier to understand?

```
def gen_primes():  
    primes = []  
    for x in range(2, 100):  
        is_prime = True  
        for y in range(3, x):  
            if x%y == 0:  
                is_prime = False  
        if is_prime:  
            primes.append(x)  
    return primes
```

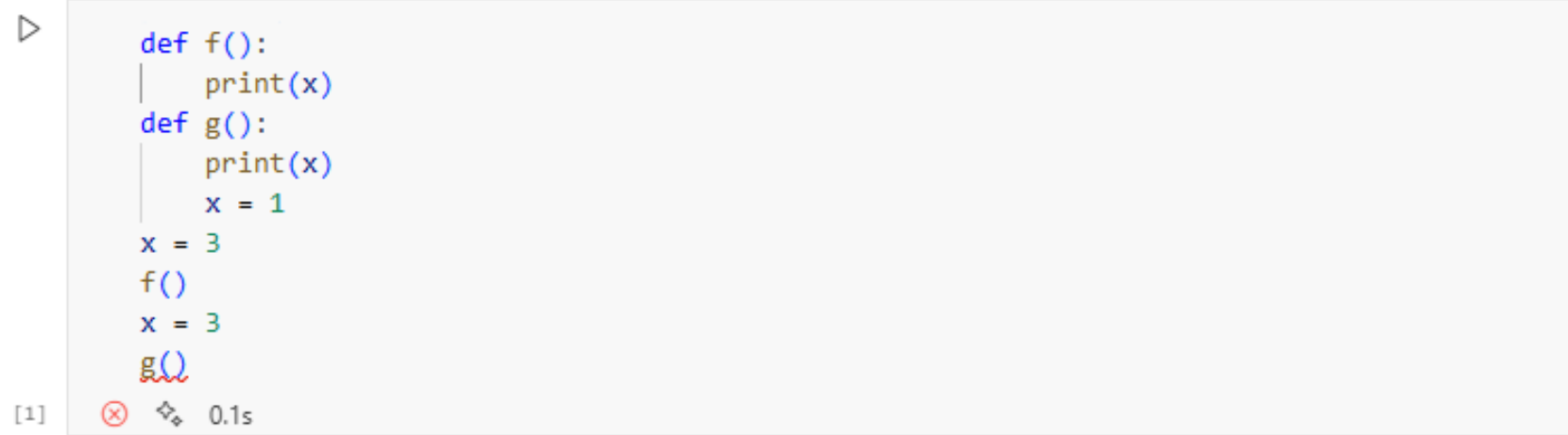
```
primes = gen_primes()  
print(primes)
```

✓ 0.0s

[2, 3, 4, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

Scope (local vs. global)

```
def f():  
    print(x)  
def g():  
    print(x)  
    x = 1  
x = 3  
f()  
x = 3  
g()
```



A Jupyter Notebook cell containing Python code. The code defines two functions, `f()` and `g()`, both of which print the value of `x`. `f()` prints `x` without modifying it. `g()` prints `x` and then assigns the value 1 to `x`. Below the function definitions, the code sets `x = 3`, calls `f()`, sets `x = 3` again, and then calls `g()`. The `g()` call is underlined in red, indicating an error. Below the code, the output shows the number 3, followed by a red dashed line and an `UnboundLocalError` traceback.

```
def f():  
    print(x)  
def g():  
    print(x)  
    x = 1  
x = 3  
f()  
x = 3  
g()
```

[1]   0.1s

... 3

...

UnboundLocalError

Traceback (most recent call last)

Cell In[1], line 9

```
7 f()  
8 x = 3  
----> 9 g()
```

Cell In[1], line 4

```
3 def g():  
----> 4     print(x)  
5     x = 1
```

UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Functions and modules

```
# Fibonacci numbers module
```

fibonacci.py

```
def fib(n):  
    """Write Fibonacci series up to n."""  
    a, b = 0, 1  
    while a < n:  
        print(a, end=' ')  
        a, b = b, a+b  
    print()  
  
def fib2(n):  
    """Return Fibonacci series up to n."""  
    result = []  
    a, b = 0, 1  
    while a < n:  
        result.append(a)  
        a, b = b, a+b  
    return result
```

Now enter the Python interpreter and import this module with the following command:

```
>>> import fibo
```

This does not add the names of the functions defined in `fibo` directly to the current [namespace](#) (see [Python Scopes and Namespaces](#) for more details); it only adds the module name `fibo` there. Using the module name you can access the functions:

```
>>> fibo.fib(1000)  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987  
>>> fibo.fib2(100)  
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]  
>>> fibo.__name__  
'fibo'
```

Functions and modules

There is a variant of the `import` statement that imports names from a module directly into the importing module's namespace. For example:

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

This does not introduce the module name from which the imports are taken in the local namespace (so in the example, `fibo` is not defined).

There is even a variant to import all names that a module defines:

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Functions and modules

If the module name is followed by `as`, then the name following `as` is bound directly to the imported module.

```
>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

This is effectively importing the module in the same way that `import fibo` will do, with the only difference of it being available as `fib`.

It can also be used when utilising [from](#) with similar effects:

```
>>> from fibo import fib as fibonacci
>>> fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Keywords

False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield

You will most likely need the following here:

False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield

Useful groupings:

from, import, as
and, or, not, False, True
if, elif, else
for, in, while, continue, break
def, return, lambda
try, except

Most things in Python are objects and can be redefined, even many built-in operators.

Keywords cannot be redefined in Python and are not objects.

Note: you can access all Python's built-in identifiers with:

```
import builtins
dir(builtins) # to view
```

In-class Python review exercise

The notebook for this lecture (session03, in-class_exercise03.ipynb) will walk you through examples of

- Variable assignments and types
- Scope
- Data structures
- Functions and modules
- (as always, many keywords)

The following slides have additional examples of keyword usage for your reference.

is, def, return, for, in, lambda

```
a = [12, 11, 10]
b = a
a is b
```

✓ 0.0s

True

```
a = [12, 11, 10]
b = [12, 11, 10]
a is b
```

✓ 0.0s

False

```
def subtract(a, b):
    return a - b
```

```
print(subtract(1, 2))
```

```
# in python most things are objects
functions = [subtract, sum]
```

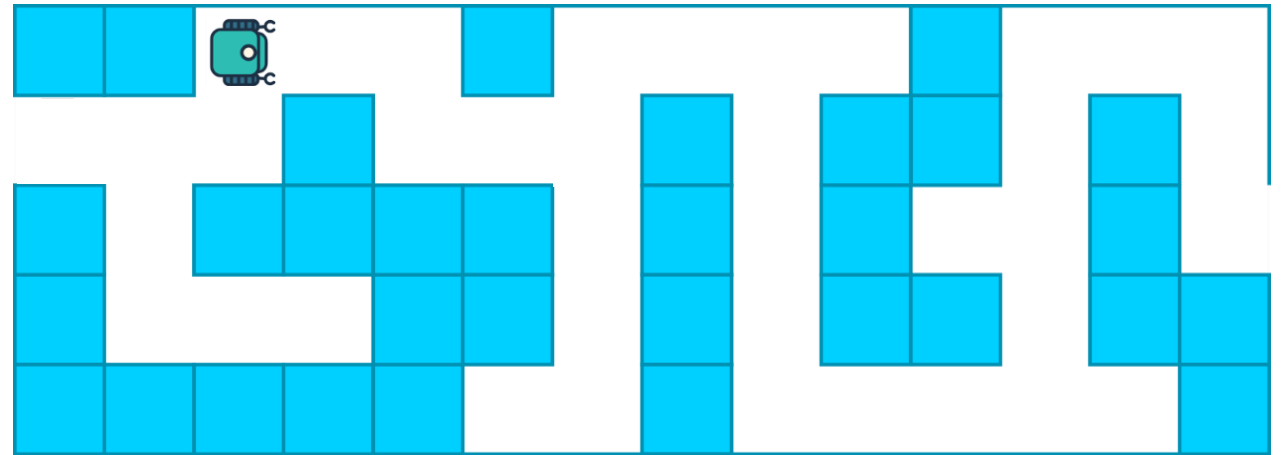
```
for func in functions:
    print(func(1, 2))
```

```
f = lambda x: x**2
f(2)
```

def, return; if, else; and, or, Boolean operators

```
def at_fourway_crossing():  
    if robot.left_is_clear() and robot.front_is_clear() and robot.right_is_clear() and  
robot.behind_is_clear():  
        return True  
    else:  
        return False
```

```
def at_corner():  
    if robot.left_is_clear() or robot.right_is_clear():  
        return True  
    else:  
        return False
```



for, break, continue, while, not

```
formula = "CH4"
```

```
for character in formula:
    print(character)

    if character.isdigit():
        break
```

```
terms = "H2O + CO2"
```

```
for character in terms:
    # Skip the space character
    if character == " ":
        continue

    print(character)
```

```
while not target_reached():
    if look_ahead():
        move_forward()
    else:
        turn_left()
```

try, except

```
def amounts_to_percentages(amounts : list) -> list:  
    total = sum(amounts)  
    normalized = []  
    for amount in amounts:  
        normalized.append(amount / total * 100)  
    return normalized
```

```
data = [11, 2, 25]  
amounts_to_percentages(data)
```

→ [28.947368421052634, 5.263157894736842, 65.78947368421053]

```
data = [0, 0]  
amounts_to_percentages(data)  
→ ZeroDivisionError
```

```
try:  
    amounts_to_percentages(data)  
except ZeroDivisionError:  
    print("Error: Cannot convert to percentages, total is zero.")
```